This CVPR paper is the Open Access version, provided by the Computer Vision Foundation.
Except for this watermark, it is identical to the accepted version;
the final published version of the proceedings is available on IEEE Xplore.

# PURA: Parameter Update-Recovery Test-Time Adaption for RGB-T Tracking

Zekai Shao* Yufan Hu* Bin Fan Hongmin Liu†
School of Intelligence Science and Technology, University of Science and Technology Beijing
melan.shao@foxmail.com, huyufanqaixuan@gmail.com, bin.fan@ieee.org, hmliu_82@163.com

## Abstract

*Maintaining stable tracking of objects in domain shift scenarios is crucial for RGB-T tracking, prompting us to explore the use of unlabeled test sample information for effective online model adaptation. However, current Test-Time Adaptation (TTA) methods in RGB-T tracking dramatically change the model's internal parameters during long-term adaptation. At the same time, the gradient computations involved in the optimization process impose a significant computational burden. To address these challenges, we propose a Parameter Update-Recovery Adaptation (PURA) framework based on parameter decomposition. Firstly, our fast parameter update strategy adjusts model parameters using statistical information from test samples without requiring gradient calculations, ensuring consistency between the model and test data distribution. Secondly, our parameter decomposition recovery employs orthogonal decomposition to identify the principal update direction and recover parameters in this direction, aiding in the retention of critical knowledge. Finally, we leverage the information obtained from decomposition to provide feedback on the momentum during the update phase, ensuring a stable updating process. Experimental results demonstrate that PURA outperforms current state-of-the-art methods across multiple datasets, validating its effectiveness. The project page is at* https://melantech.github.io/PURA.

## 1. Introduction

RGB-T tracking is an object-tracking method that combines RGB and thermal imaging information and is widely used in object tracking in complex environments. During the tracking process, due to the presence of different scenarios (such as various backgrounds and light variations), the problem of data distribution shift [9, 24, 36] is inevitably encountered during the testing process. However, the parameters of traditional trackers are usually fixed during the inference stage. Moreover, the limitations of training data make RGB-T trackers extremely prone to overfitting. These problem gives rise to a crucial question: can the RGB-T tracker effectively utilize sample information to improve its capabilities during the tracking process? Notably, the lack of annotation information makes it unrealistic to update the model through supervised learning [11, 13, 42, 53]. Furthermore, object-tracking scenarios in complex environments usually have real-time requirements, and supervised learning will introduce extensive computational delays, causing the tracker to fail to output results promptly.

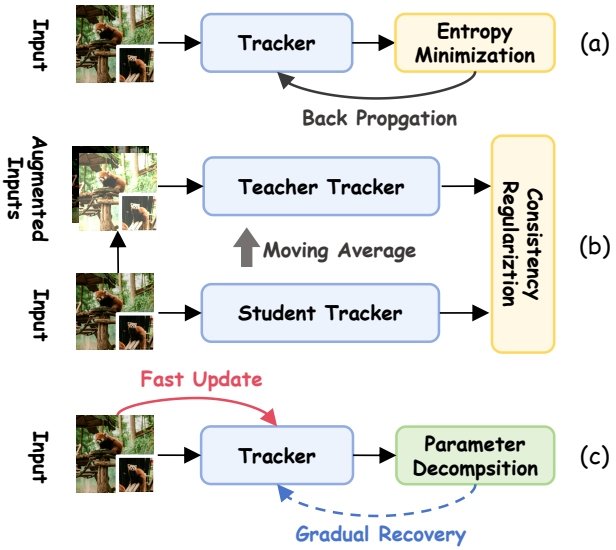



Figure 1. Comparison between our proposed Parameter Update-Recovery Adaption (PURA) framework with existing test-time adaption methods. (a) Entropy minimization-based method. (b) Consistency regularization-based method. (c) Our proposed PURA framework.

In recent years, test-time adaptation (TTA) has been used for online model adaption. During the testing phase, TTA can dynamically adjust the model parameters, enabling the model to adapt to new distributions without the need to access the original training data. Notably, TTA has unique advantages in RGB-T tracking scenarios because it can achieve real-time adaptation to distribution changes caused by different scenarios, thereby enhancing the stability and

*Equal contribution
†Corresponding author

robustness of the tracking process. Through simple modifications, existing TTA methods can be integrated into trackers. However, there is currently no proposed solution for RGB-T tracking. As shown in Figure 1 (a), the entropy minimization method [12, 34, 35, 39] adjusts the affine parameters of the BatchNorm layer [17] in the pre-trained model by minimizing the average entropy of the test batches. However, in long-term tracking scenarios, frequent updates will cause drastic changes in model parameters, making the model unable to effectively retain previously learned knowledge. In addition, as shown in Figure 1 (b), methods based on consistency regularization [2, 41, 47, 51] increase a large number of model inputs, seriously increasing the computational burden. Meanwhile, the learning process based on gradient descent usually involves high computational complexity, significantly impacting the model's operational efficiency. Therefore, exploring a method that can ensure model performance while reducing computational cost remains a critical problem that urgently needs to be solved.

To address the above problems effectively, we propose an innovative Parameter Update-Recovery Adaptation (PURA) framework based on pseudo-gradient decomposition. As shown in Figure 1 (c), this framework mainly consists of two key components: fast parameter update and parameter decomposition recovery. The fast parameter update plays a crucial role. It updates a small number of parameters in the BatchNorm layer by calculating the mean and variance of test samples. This operation can efficiently capture new domain features, enabling the knowledge in the existing model to adapt to new environmental conditions. It is worth noting that this update calculation method has a meager cost, firmly ensuring the efficiency of the update process. Although the fast parameter update module has significant advantages, since its update process is unsupervised, frequent update operations will inevitably lead to the gradual accumulation of parameter miscalibration problems. Therefore, we suggest decomposing the parameter trajectory to achieve parameter recovery. We regard the trajectory of parameter changes as pseudo-gradients. By orthogonally decomposing the pseudo-gradient, we can decompose it into the principal direction of parameter updates and their update magnitudes. Through an in-depth analysis of decomposed information, we can accurately identify the principal direction of updates from pseudo-gradients filled with a large amount of noise information and recover the parameters in that direction. In addition, by conducting consistency analysis on the update magnitude, we can provide dynamic feedback for the parameter update process to ensure a more stable update process.

Our main contributions are as follows:

- We propose a parameter update-recovery adaption framework that utilizes the statistical characteristics of test samples for fast domain adaptation while performing orthogonal decomposition on parameter trajectories to gradually recover critical parameters in the principal update direction, thereby alleviating the domain shift problem while maintaining model stability.
- We introduce adaptive momentum scaling that provides feedback on the parameter update process based on the consistency of update magnitude, ensuring its stability.
- Experimental results on three popular benchmark datasets demonstrate that our approach exhibits exceptional adaptability, outperforming existing methods.

## 2. Related Work

**RGB-T Tracking.** In most cases, the existing single modal trackers [1, 5, 10, 45] perform well. However, their performance must improve to meet the requirements under challenging conditions such as low light and extreme weather. RGB-T tracking methods combine complementary information from RGB and thermal imaging images to address this issue, enhancing tracking robustness. Some methods [16, 22, 28, 48] emphasize the effective fusion of information from two modalities. CAT [22] proposed a novel challenge-aware framework with parallel and hierarchical architecture, embedding modal sharing and specific challenges as residual information into the backbone. TBSI [16] designed a method to fuse modal features through the interaction between templates and search areas, thereby improving tracking performance. In addition, some recent models based on fine-tuning paradigms [4, 14, 40, 54] have significantly improved performance while reducing trainable parameters. ViPT [54] freezes the pre-trained base model and only trains a small number of parameters for modality interaction, achieving state-of-the-art performance while ensuring parameter efficiency. BAT [4] employs a dual-stream architecture and introduces a general bi-directional adapter for cross-feature prompting, further enhancing tracking performance based on fine-tuning paradigms. Although the methods above have achieved significant progress in model training, they usually need to fully consider the impact of distribution shift on model performance. Our proposed method introduces TTA into the RGB-T tracking, thereby alleviating the negative impact of environmental changes on trackers.

**Test-Time Adaption.** Test-time adaptation (TTA) aims to enhance model performance on test data against distribution shifts. Methods based on batch normalization calibration [29, 33, 36, 52] leverage the statistical features of test samples and employ an exponential moving average (EMA) strategy to recalibrate the BatchNorm layer, thereby improving robustness under distributional changes. Building on this, DELTA [52] records the proportions of each category using EMA to weight samples effectively, accommodating imbalanced data. Entropy minimization

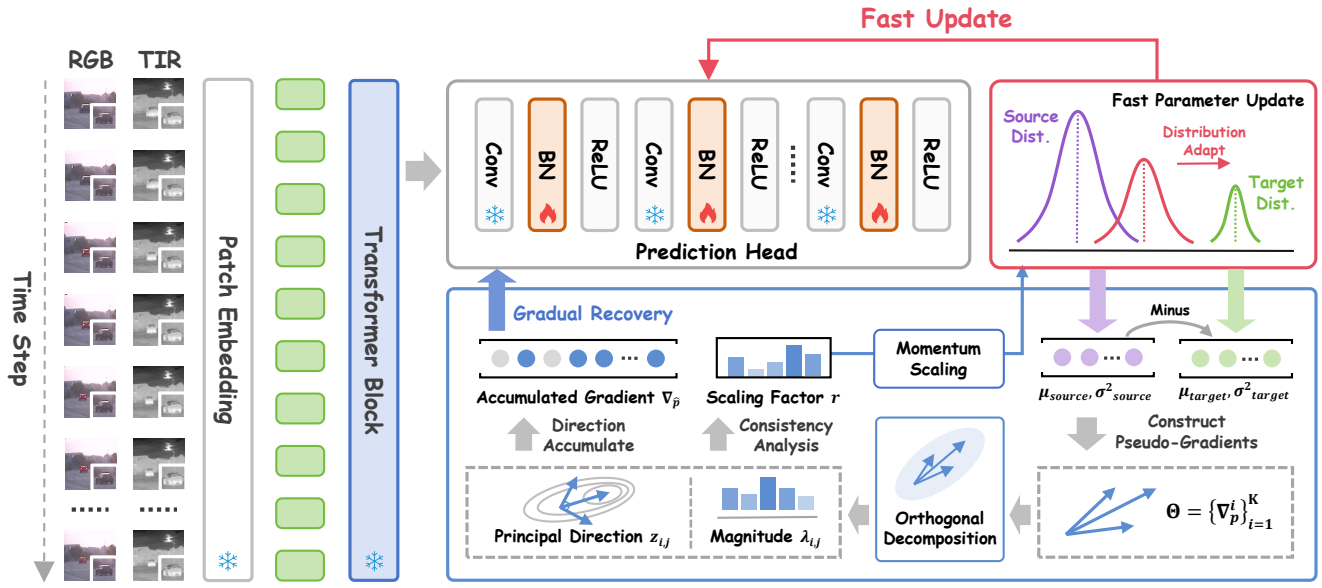



Figure 2. Overview of Parameter Update-Recovery Adaption (PURA). It consists of two key components: fast parameter update and parameter decomposition recovery. In the parameter updating phase, we use test data to adjust a small number of statistical parameters in the BatchNorm layer, effectively capturing new distribution features. In the parameter recovery phase, we analyze the parameter trajectory through singular value decomposition to isolate important parameters that align with the principal update direction for recovery.

[12, 34, 35, 39] is a robust technique for unlabeled test-time adaptation. Tent [39] is a novel approach that adjusts the affine parameters of the BatchNorm layer in pre-trained models by minimizing the average entropy of test batches. SAR [35] aims to guide the model to be located in a flat area of the entropy loss surface. Inspired by the mean teacher [38] framework, some studies [2, 41, 47, 51] utilize pseudo labels obtained from weak data augmentation via teacher networks to regularize consistency. CoTTA [41] employs multiple enhancement techniques to optimize the pseudo labels generated by teacher networks. Meanwhile, LAME [2] leverages adjacent consistency to ensure that the assignments of neighboring points in the feature space remain uniform. While these methods perform well in various tasks, most are unsuitable for RGB-T tracking. The batch normalization calibration method has an error accumulation problem, while the entropy minimization and consistency regularization methods will generate relatively high computational complexity. Considering these limitations and inspired by FSTTA [12], which efficiently decomposes gradients step by step, our proposed method focuses on achieving a balance between fast updates and parameter stability.

## 3. Method

This work proposes a Parameter Update-Recovery Adaption (PURA) framework, a test-time adaptation framework for RGB-T tracking. This method efficiently addresses data distribution shifts by updating the statistical parameters of the BatchNorm [17] layer using test samples. Additionally, to alleviate overfitting caused by frequent updates, we employ singular value decomposition to recover critical parameters along the main update direction, thereby enhancing parameter stability. Finally, we dynamically adjust the update momentum by utilizing the consistency information of the pseudo-gradients in the principal direction. The overall framework of the proposed method is illustrated in Figure 2.

### 3.1. Fast Parameter Update

Batch Normalization [17] is a widely used technique in deep neural networks. It normalizes each mini-batch of data, ensuring that the distribution of inputs to each layer remains relatively stable throughout the training process. This stability mitigates issues such as vanishing and exploding gradients, facilitating faster convergence and enhancing overall model performance. Specifically, for each batch of training data $(x_b, y_b) \sim D^{train}(x, y)$, the batch normalization layer normalizes the data using the following formula:

$$\hat{x}_b = \gamma \cdot \frac{x_b - \mathbb{E}[X]}{\sqrt{\mathrm{Var}[X] + \epsilon}} + \beta, \tag{1}$$

where $\gamma$ and $\beta$ represent trainable affine parameters, and $\epsilon$ is used to ensure the stability of numerical calculations. The expected values $\mathbb{E}[X] \in \mathbb{R}^{\mathrm{D}}$ and $\mathrm{Var}[X] \in \mathbb{R}^{\mathrm{D}}$ are estimated values obtained through EMA during the training process, where D represents the dimension of parameters.

Generally, test data $(x_t, y_t) \sim S^{\text{test}}(x, y)$ and training data $S^{\text{train}}$ are assumed to be identically distributed. During inference, the model directly normalizes the batch data us-

ing the mean and variance estimated during training, ensuring that the model generates a deterministic output. However, maintaining distributional consistency between training and test data is unrealistic. Such distributional disparity causes domain shift, impeding the model's adaptation to new data. Limited training datasets also make the tracker prone to overfitting. Thus, adjusting the training-stage estimated mean and variance with test data during inference can realign the activation distributions between training and test, enabling the model to recalibrate internal parameters and better accommodate new data characteristics.

Our approach uses the mean and variance estimated during training as initial parameters and continuously updates them with test samples to fit the distribution of the test data with minimal computational overhead. Specifically, for the input test data $x_t$, we first calculate its mean $\mu_t = \sum x_t/\mathrm{N}$ and variance $\sigma_t^2 = \sum(x_t - \mu_t)^2/\,(\mathrm{N}\text{-}1)$, where N represents the number of elements of each batch of test data. Subsequently, we continuously update the mean and variance using the measurements derived from the current test data. Notably, we use EMA to update parameters, which can use the mean and variance obtained from the training stage as priors to provide a reference and starting point for the update while reducing the impact of noise on the update process. The above process can be represented as:

$$\begin{aligned} \hat{\mu}_t &= (1-m)\cdot\hat{\mu}_{t-1} + m\cdot\mu_t, \\ \hat{\sigma}_t^2 &= (1-m)\cdot\hat{\sigma}_{t-1}^2 + m\cdot\sigma_t^2, \end{aligned} \tag{2}$$

here, $\hat{\mu}$ and $\hat{\sigma}^2$ are the estimated mean and variance during inference, while $\mu$ and $\sigma^2$ represent the mean and variance of each batch of data. $t$ denotes the time step, $m$ is the momentum parameter for $\mu$ and $\sigma^2$.

### 3.2. Parameter Decomposition for Recovery

During the testing phase, updating model parameters can adapt the model to domain shifts. However, [34, 41] note that long-term model adaptation causes noise and error accumulation. In tracking tasks, significant distribution shifts can miscalibrate the tracker. Since parameters cannot be recovered quickly, the tracker keeps using the wrong parameters in later frames, restricting its performance. To tackle this, we propose parameter decomposition recovery. It aims to find the main update direction via the parameter update trajectory and then restore the model parameters.

We conduct model recovery every K frames. During each model update, the difference between the updated and the original parameter can reflect the direction and magnitude of the parameter change and thus can be regarded as pseudo-gradient, which can be expressed as $\nabla_p = \left\{\hat{\mu} - \mu_{t-1}, \hat{\sigma}^2 - \sigma_{t-1}^2\right\} \in \mathbb{R}^{2\times\mathrm{D}}$. After that, the pseudo-gradients of the model's parameters are collected into a set $\mathbf{\Theta} = \left\{\nabla_p^i\right\}_{i=1}^{\mathrm{K}} \in \mathbb{R}^{2\times\mathrm{D}\times\mathrm{K}}$ to represent the trajectory of parameter changes. Intuitively, parameter recovery can be achieved by adding pseudo-gradients on the parameters, but this is equivalent to completing the recovery by saving historical parameters. We note that the pseudo-gradients consistent with the principal update direction usually can reflect the data's core features and main change patterns. Frequently updating the parameters corresponding to these pseudo-gradients will cause the model to overfit, which damages its original capabilities. Therefore, using these pseudo-gradients to recover their corresponding parameters helps the model retain critical knowledge. Inspired by principal component analysis [43], the principal update direction and its magnitude of the pseudo-gradient set can be found through SVD:

$$\lambda_{i,j}, z_{i,j} = \mathbf{SVD}\left(\frac{1}{\mathrm{K}-1}\bar{\mathbf{\Theta}}^\top\bar{\mathbf{\Theta}}\right), \tag{3}$$

where $\lambda_{i,j}$ and $z_{i,j}$ represent the eigenvalue and its corresponding eigenvector, and the two correspond to the principal direction and magnitude of the pseudo-gradients. $j$ represents the parameter index. These vectors are arranged in descending order of importance. $\bar{\mathbf{\Theta}}$ represents the centralized pseudo-gradient matrix obtained by removing the mean. To preserve the original information of the pseudo-gradients, we map the feature vectors back to the original space and perform normalization to ensure numerical stability. However, normalization may lead to numerical rigidity and affect the update process. Thus, we introduce a reference value for alignment correction. We obtain this reference value by calculating the parameter variation $\nabla\theta = \left\{\mu_{t-\mathrm{K}} - \mu_t, \sigma_{t-\mathrm{K}}^2 - \sigma_t^2\right\} \in \mathbb{R}^{2\times\mathrm{D}}$, and use its norm to achieve adaptive length calibration of pseudo-gradients:

$$\hat{z}_{i,j} = \mathcal{N}_2\left(\bar{\mathbf{\Theta}}_i^\top z_{i,j}\right)\cdot\|\nabla\theta\|_2, \tag{4}$$

where $\mathcal{N}_2(\cdot)$ represents $\mathrm{L}_2$ normalization. The pseudo-gradient vector retains important information and enhances computational stability through normalization and scaling. However, these vectors do not have clear directionality, which may lead to confusion in the direction of parameter recovery. Therefore, we assess the directional correlation by computing the inner product between $\nabla\theta_j$ and $\hat{z}_{i,j}$ and calibrate the direction of each vector using the similarity evaluation function. Considering that the eigenvalues $\lambda_{i,j}$ represent the importance of each principal component, we also utilize the normalized eigenvalues to aggregate the principal components, resulting in a more robust pseudo-gradient vector. The above process can be expressed as:

$$\nabla_{\hat{p}} = \sum_{i=1}^{\mathrm{K}}\left(\mathcal{N}_2\left(\lambda_{i,j}\right)\cdot\delta\left(\nabla\theta_j\cdot\hat{z}_{i,j}\right)\cdot\hat{z}_{i,j}\right), \tag{5}$$

where $\delta(\cdot)$ represents the similarity evaluation function. When the input value is greater than or equal to 0, the func-

tion value is 1. Otherwise, it is -1. Based on the aforementioned pseudo-gradient, we perform parameter recovery every K frames:

$$\begin{aligned} \hat{\mu}_t &= \hat{\mu}_{t-1} + \eta_\mu \cdot \nabla_{\hat{p}}^{(\mu)}, \\ \hat{\sigma}_t^2 &= \hat{\sigma}_{t-1}^2 + \eta_\sigma \cdot \nabla_{\hat{p}}^{(\sigma)}, \end{aligned} \tag{6}$$

here, $\eta_\mu$ and $\eta_\sigma$ represent the learning rates for the mean and variance, respectively. Notably, we observe significant differences in the mean and variance numerical ranges. Therefore, using different learning rates enables a more precise recovery process.

### 3.3. Adaptive Momentum Scaling

**Obtain Scaling Factor for Momentum.** We propose adaptive momentum scaling that adjusts the model's momentum during the parameter update phase by judging the consistency of the principal update direction. Considering that the eigenvalue $\lambda_i$ represents the variance of each principal component, indicating the importance of various principal update directions, we can use the exponential moving average of the sum of the eigenvalues $\lambda = \sum \lambda_i$ as a reference value for updating the direction consistency of parameters:

$$\bar{\lambda} = (1 - m_\lambda) \cdot \bar{\lambda} + m_\lambda \cdot \lambda, \tag{7}$$

here, $m_\lambda$ represents the momentum parameter for $\bar{\lambda}$. The scaling factor $r$ is determined by the difference between $\lambda$ and $\bar{\lambda}$. Consistent update directions result in a smaller difference, allowing a larger $r$, and vice versa. The scaling factor $r$ is calculated as:

$$r = \mathcal{T}(\xi - |\lambda - \bar{\lambda}|), \tag{8}$$

where $\xi$ is the threshold parameter, and $\mathcal{T}(\cdot)$ is the truncation operation to constrain $r$ within a reasonable range.

**Momentum Scaling and Model Updating.** We obtain the updated momentum by calculating $\hat{m} = \mathcal{T}(r \cdot m)$, and use the updated momentum for learning in the next time step:

$$\begin{aligned} \hat{\mu}_t &= (1 - \hat{m}) \cdot \hat{\mu}_{t-1} + \hat{m} \cdot \mu_t, \\ \hat{\sigma}_t^2 &= (1 - \hat{m}) \cdot \hat{\sigma}_{t-1}^2 + \hat{m} \cdot \sigma_t^2. \end{aligned} \tag{9}$$

When updating the momentum, we also employ truncation to prevent excessively large or small momentum from causing unstable updates.

## 4. Experiments

### 4.1. Datasets and Evaluation Metrics

We quantitatively evaluate various methods on three popular RGB-T tracking datasets, GTOT [19], RGBT210 [20], and RGBT234 [21], using precision rate (PR) and success rate (SR) as evaluation metrics. PR measures the accuracy of target position predictions by calculating the percentage of frames where the predicted center position falls within a specified threshold distance from the ground truth. In contrast, SR is assessed based on the percentage of frames where the Intersection over Union (IoU) between the predicted bounding box and the ground truth bounding box exceeds a specified threshold. Notably, in the RGBT234 and GTOT datasets, the ground truth for RGB and infrared images does not fully align. We use maximum precision rate (MPR) and maximum success rate (MSR) as evaluation metrics instead of PR and SR to ensure a fair comparison.

### 4.2. Implementation Details

We conduct all experiments on two NVIDIA A40 GPUs. Our approach uses only BatchNorm layers in the prediction head for parameter updates. We set the parameter recovery interval K $= 3$ and the base momentum for updates $m = 1 \times 10^{-1}$. The learning rates for the mean and variance of BatchNorm layers are $\eta_\mu = 1 \times 10^{-4}$ and $\eta_\sigma = 5 \times 10^{-5}$ respectively. We empirically set the variance threshold for adaptive momentum scaling to $\xi = 1.6$ and the update momentum to $m_\lambda = 0.95$. The truncation interval for scaling factor $r$ is [0.9, 1.1], and momentum $m$ is in [0.1, 0.11]. Our baseline uses a one-stream ViT-B [6] backbone with a template update mechanism, concatenating token sequences of two modalities for joint feature extraction. A convolutional head [45] processes search region features for results, and a score prediction head [5] updates dynamic templates based on confidence and a threshold.

### 4.3. Comparison with State-of-the-art Trackers

**Comparison with Competitive RGB-T Trackers.** In the RGBT234, RGBT210, and GTOT datasets, we conduct a comprehensive comparison of our proposed PURA tracker against state-of-the-art (SOTA) trackers, with detailed results presented in Table 1. Specifically, on the RGBT234 dataset, PURA achieves 93.3% in MPR and 70.3% in MSR, outperforming several SOTA pre-training methods. Compared to TBSI [16], PURA shows an improvement of 5.3% in MPR and 4.5% in MSR, underscoring its superior accuracy and robustness. On the RGBT210 dataset, PURA demonstrates outstanding performance, achieving 90.3% in PR and 66.8% in SR, ranking highest among all compared methods. Relative to its closest competitor, MambaVT [18], PURA achieves an increase of 1.8% in PR and 2.4% in SR, highlighting its competitive advantage in tracking accuracy and reliability. Despite the relatively short sequences in the GTOT dataset, which limit PURA's capability for extensive updates via continuous learning, it still attains 95.7% in MPR and 78.6% in MSR. Notably, PURA excels in processing speed, reaching 42 FPS, thus ensuring real-time performance without compromising tracking quality. These results underscore the comprehensive and exceptional perfor-

Table 1. Comparison of state-of-the-art performance across the GTOT, RGBT210, and RGBT234 datasets. "SOT" refers to pre-training on the combined splits of COCO [26], LaSOT [7], GOT-10k [15], and TrackingNet [32], a standard practice for training SOT tracking methods. "Online" and "Offline" represent whether the tracker can access the test data. **Bold** denotes the highest performance.

| | Method | Pretrain | GTOT [19] | | RGBT210 [20] | | RGBT234 [21] | | FPS↑ |
|---|---|---|---|---|---|---|---|---|---|
| | | | MPR(%)↑ | MSR(%)↑ | PR(%)↑ | SR(%)↑ | MPR(%)↑ | MSR(%)↑ | |
| Offline | LANet [8], TMM'23 | ImageNet | 91.3 | 75.1 | 77.9 | 56.7 | 79.5 | 58.4 | 21.7 |
| | ViPT [54], CVPR'23 | SOT | - | - | - | - | 83.5 | 61.7 | - |
| | TBSI [16], CVPR'23 | SOT | - | - | 85.1 | 62.5 | 88.0 | 65.8 | 36.2 |
| Online | HMFT [49], CVPR'22 | - | 91.2 | 74.9 | 78.6 | 53.5 | 78.8 | 56.8 | 30.2 |
| | CMD [50], CVPR'23 | - | 89.2 | 73.4 | - | - | 82.4 | 58.4 | 30.0 |
| | MMMPT [3], IPT'23 | - | 90.3 | 73.6 | - | - | 79.8 | 54.6 | 19.3 |
| | APFNet [44], AAAI'22 | ImageNet | 90.5 | 73.7 | 79.9 | 54.9 | 82.7 | 57.9 | 1.3 |
| | DRGCNet [30], SENS'22 | ImageNet | 90.5 | 73.5 | - | - | 82.5 | 58.1 | 4.9 |
| | CAT++ [27], TIP'24 | ImageNet | 91.5 | 73.3 | 82.2 | 56.1 | 84.0 | 59.2 | 14.0 |
| | LMINet [31], SENS'24 | ImageNet | 91.4 | 73.6 | - | - | 83.8 | 59.2 | 5.6 |
| | MPLT [28], arXiv'23 | SOT | - | - | 86.2 | 63.0 | 88.4 | 65.7 | 22.8 |
| | TATrack [40], AAAI'24 | SOT | - | - | 85.3 | 61.8 | 87.2 | 64.4 | 26.1 |
| | STMT [37], arXiv'24 | SOT | - | - | 83.0 | 59.5 | 86.5 | 63.8 | 39.1 |
| | CFBT [46], arXiv'24 | SOT | - | - | 87.7 | 63.0 | 89.9 | 65.9 | - |
| | MambaVT [18], arXiv'24 | SOT | 95.2 | 78.6 | 88.5 | 64.4 | 90.7 | 67.5 | 31.1 |
| | PURA | SOT | **95.7** | **78.6** | **90.3** | **66.8** | **93.3** | **70.3** | **42.0** |

Table 2. Comparison of our method with other TTA methods under different trackers in the LasHeR→RGBT234 scenario.

| Method | MPR↑ | MSR↑ | FPS↑ | Method | MPR↑ | MSR↑ | FPS↑ |
|---|---|---|---|---|---|---|---|
| TBSI [16] | 85.3 | 62.5 | 40.2 | ViPT [54] | 83.5 | 61.7 | 41.3 |
| +Tent [39] | 85.8 | 64.2 | 12.7 | +Tent [39] | 84.1 | 62.6 | 13.1 |
| +ETA [34] | 85.8 | **64.5** | 37.3 | +ETA [34] | 83.9 | 62.5 | **39.2** |
| +AdaBN [25] | 85.9 | 63.8 | **37.6** | +AdaBN [25] | 83.9 | 62.5 | 36.7 |
| +PURA | **86.3** | 63.8 | 33.2 | +PURA | **84.7** | **62.9** | 33.5 |

mance of PURA, showcasing its adaptability and efficient tracking capabilities across diverse scenarios.

**Analysis of Different Trackers.** To further validate the effectiveness of our proposed PURA method, we conducted experiments on two representative trackers (TBSI using weights trained on LasHeR) using different TTA approaches, as shown in Table 2. Experimental results demonstrate that when using TBSI as the base tracker, PURA achieved the highest MPR of 86.3%. Meanwhile, for another advanced tracker, ViPT, PURA also demonstrated superior performance with an MPR of 84.7% while maintaining a processing speed of 33.5 FPS. These results demonstrate that our method is adaptable and stable in different application scenarios.

**Analysis Across Various Attributes.** The RGBT234 dataset has 12 unique attributes for evaluating PURA's performance against state-of-the-art methods. Results in Figure 3 highlight the outstanding performance of PURA across all tested attributes, validating its effectiveness and superior adaptability to various scenarios. Under a range of extreme conditions, our model significantly outperforms its

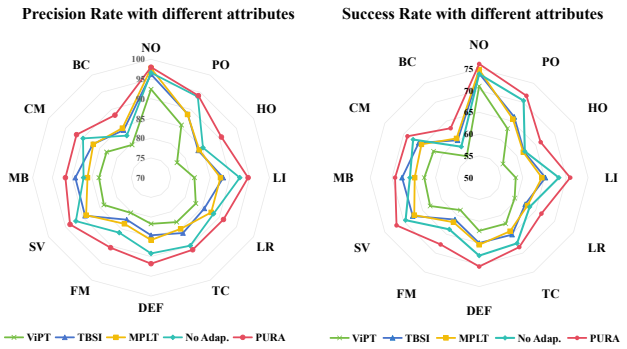



Figure 3. Comparison between PURA and competing methods across various attributes in the LasHeR→RGBT234 scenario.

competitors. Notably, in the "HO" attribute, our PR score surpasses the no-adaption approach by 5.4%. These results further prove the effectiveness of PURA in addressing domain shifts in complex environments, thereby significantly enhancing the model's generalization capability.

### 4.4. Domain Shift in RGB-T Tracking

**Domain Shift.** To evaluate the performance of PURA across different domain shift scenarios, we adopt three settings in Table 3: LasHeR→GTOT, LasHeR→RGBT210, and LasHeR→RGBT234. The proposed method is compared with several TTA methods, including three gradient-based optimization methods: Tent [39], ETA [34], and CoTTA [41], as well as an optimization-free method, AdaBN [25]. In the LasHeR→GTOT scenario, CoTTA requires repeated inference on different augmented samples

Table 3. Performance comparison in three scenarios. We compare our proposed method against optimization-free (†) and gradient-based optimization (∗) methods.

| Method | Source | Target | (M)PR↑ | (M)SR↑ | FPS↑ |
|---|---|---|---|---|---|
| No Adap. | LasHeR | GTOT | 95.1 | 78.2 | 59.5 |
| +Tent* [39] | LasHeR | GTOT | **96.0** | **78.9** | 18.5 |
| +ETA* [34] | | | 95.6 | 78.7 | 56.5 |
| +CoTTA* [41] | | | 93.5 | 77.5 | 2.8 |
| +AdaBN† [25] | | | 94.7 | 77.9 | **58.3** |
| PURA† | LasHeR | GTOT | 95.7 | 78.6 | 42.0 |
| No Adap. | LasHeR | RGBT210 | 88.6 | 65.1 | 59.5 |
| +Tent* [39] | LasHeR | RGBT210 | 89.9 | **66.8** | 18.5 |
| +ETA* [34] | | | 89.6 | 66.4 | 56.5 |
| +CoTTA* [41] | | | 88.4 | 65.4 | 2.8 |
| +AdaBN† [25] | | | 89.2 | 66.0 | **58.3** |
| PURA† | LasHeR | RGBT210 | **90.3** | **66.8** | 42.0 |
| No Adap. | LasHeR | RGBT234 | 90.8 | 67.6 | 59.5 |
| +Tent* [39] | LasHeR | RGBT234 | 91.8 | 69.3 | 18.5 |
| +ETA* [34] | | | 92.3 | 69.7 | 56.5 |
| +CoTTA* [41] | | | 91.2 | 68.6 | 2.8 |
| +AdaBN† [25] | | | 91.6 | 69.1 | **58.3** |
| PURA† | LasHeR | RGBT234 | **93.3** | **70.3** | 42.0 |

(reduced augmentations from 32 to 8 per frame for speed) for each frame, significantly increasing the computational burden and severely limiting inference speed. In contrast, PURA achieves 95.7% of MPR and 78.6% of MSR, second only to Tent at 96.0% and 78.9%, respectively. PURA demonstrates faster speed, reaching 42 FPS, significantly better than Tent, showcasing a good balance between efficiency and performance. However, the speed of PURA is not as fast as that of AdaBN and ETA. The former is because AdaBN does not require parameter recovery during inference, thus avoiding additional computational overhead. The latter is because the reduced number of inference batches affects the sample selection mechanism of ETA, making it impossible to update in most cases. PURA achieves a good balance between efficiency and performance. In the LasHeR→RGBT210 scenario, PURA achieves 90.3% of MPR and 66.8% of MSR, comparable to Tent at 89.9% of MPR and 66.8% of MSR. However, regarding inference speed, PURA still surpasses Tent, further evidencing its efficiency. Additionally, the compromises of precision between ETA and AdaBN result in overall performance inferior to that of PURA. In the LasHeR→RGBT234 scenario, the performance of PURA is particularly noteworthy, achieving 93.3% of MPR and 70.3% of MSR, which surpasses Tent at 91.8% and 69.3% and establishes PURA as the highest-performing method among all.

**Dataset Corruption.** To further verify PURA's effectiveness in extreme conditions, we apply three corruption levels to the LasHeR testing set [23] to mimic adverse situations. Specifically, we modify the brightness, contrast, saturation, and hue of original images differently, with detailed parameter settings in the supplementary materials. Table 4 shows the performance comparison between PURA and the no-adaptation method under various corruption severities. PURA surpasses the no-adaptation approach at all three levels. In the most severe condition (Severity 3), PURA reaches 44% of MPR, a 3.9% improvement over the no-adaptation method. Notably, PURA's advantage over the no-adaptation approach grows as data corruption worsens. The results further prove PURA's significant advantages in dealing with extreme data.

Table 4. Performance comparison of our method on the LasHeR testing set under different levels of data corruption.

| Method | Severity 1 | | Severity 2 | | Severity 3 | |
|---|---|---|---|---|---|---|
| | MPR | MSR | MPR | MSR | MPR | MSR |
| No Adap. | 61.4 | 49.5 | 51.6 | 42.1 | 40.1 | 32.9 |
| PURA | **62.4** | **49.7** | **53.0** | **42.5** | **44.0** | **35.2** |

Table 5. Ablation study of component effectiveness on the LasHeR→RGBT234 scenario.

| FPU | PDR | AMS | MPR(%)↑ | MSR(%)↑ |
|---|---|---|---|---|
| - | - | - | 90.8 | 67.6 |
| ✓ | - | - | 91.6 | 69.2 |
| ✓ | ✓ | - | 92.3 | 69.6 |
| ✓ | ✓ | ✓ | **93.3** | **70.3** |

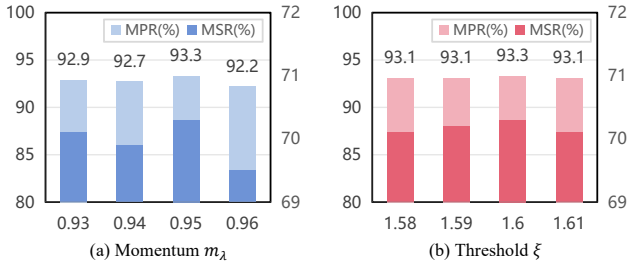
## 4.5. Ablation Studies

**Component Analysis.** Table 5 reports an ablation on the effectiveness of components within the LasHeR→RGBT234 scenario. It lists the impact of various module combinations on model performance, including Fast Parameter Update (FPU), Parameter Decomposition Recovery (PDR), and Adaptive Momentum Scaling (AMS). The results indicate that the baseline model (without any modules) achieves an MPR of 90.8% and an MSR of 67.6%. Adding FPU boosts MPR to 91.6% and MSR to 69.2%, showing its role in reducing distribution shift. Incorporating PDR further raises MPR to 92.3%, indicating it helps with knowledge forgetting from long-term updates. Finally, when all modules are applied simultaneously, MPR reaches 93.3%, and MSR increases to 70.3%, achieving the best performance. This result underscores the complementarity among the modules, indicating that their combined effects significantly enhance the overall capability of the model.

**Momentum Truncation Range.** We conduct an ablation study on the RGBT234→LasHeR scenario for different momentum truncation ranges. As shown in Table 6, when the truncation range is $[0.10, 0.11]$, the model exhibits optimal performance, achieving 93.3% of MPR and 70.3% of MSR. Shifting the range left or right reduces performance. Results indicate that excessively large or small momentum values

Table 6. Analysis of the impact of momentum truncation range on model performance.

| Range | | LasHeR→RGBT234 | |
|---|---|---|---|
| Start | End | MPR(%)↑ | MSR(%)↑ |
| 0.09 | 0.10 | 92.7 | 70.0 |
| 0.10 | 0.11 | **93.3** | **70.3** |
| 0.10 | 0.12 | 93.0 | 70.1 |
| 0.11 | 0.12 | 91.9 | 69.3 |




(a) Momentum $m_\lambda$ (b) Threshold $\xi$

Figure 4. Ablation studies on the LasHeR→RGBT234 scenario assessing the effects of different recovery intervals.

Table 7. Analysis of different learning rates for recovery on the LasHeR→RGBT234 scenario.

| $\eta_\mu$ | $\eta_\sigma$ | MPR(%)↑ | MSR(%)↑ |
|---|---|---|---|
| $1\times10^{-2}$ | $5\times10^{-5}$ | 93.0 | 70.1 |
| $1\times10^{-3}$ | $5\times10^{-5}$ | 92.9 | 70.0 |
| $1\times10^{-5}$ | $5\times10^{-5}$ | 93.2 | 70.2 |
| $1\times10^{-4}$ | $5\times10^{-2}$ | 92.9 | 69.9 |
| $1\times10^{-4}$ | $5\times10^{-3}$ | 92.2 | 69.6 |
| $1\times10^{-4}$ | $5\times10^{-4}$ | 93.2 | 70.3 |
| $1\times10^{-4}$ | $5\times10^{-5}$ | **93.3** | **70.3** |

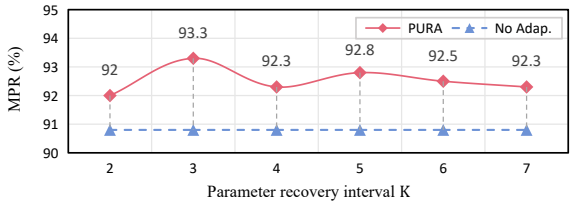



Figure 5. Ablation studies on the LasHeR→RGBT234 scenario assessing the effects of different recovery intervals.

can lead to instability in the learning process, affecting the model's overall performance.

**Analysis of Different $m_\lambda$ and $\xi$.** Figure 4 (a) demonstrates the impact of different momentum values $m_\lambda$ on performance. When $m_\lambda$ is small, the reference value $\bar{\lambda}$ becomes overly sensitive to changes, making it difficult to capture stable values. Conversely, when $m_\lambda$ is large, $\bar{\lambda}$ becomes too sluggish to respond promptly to adjustments in the update direction. Experimental results indicate that optimal momentum values allow the model to maintain good stability while retaining sufficient flexibility to adapt to changes. Figure 4 (b) investigates the impact of the threshold $\xi$ on performance in adaptive momentum scaling. When $\xi = 1.60$, the model achieves its best performance, with MPR reaching 93.3% and MSR reaching 70.3%. Slight adjustments to $\xi$, either increasing or decreasing, decrease both MPR and MSR. Therefore, the precise selection of the threshold is crucial for optimizing performance.

**Analysis of Different Learning Rates for Recovery.** Table 7 clearly shows that different learning rate combinations substantially impact model performance. Lower learning rates can result in the model parameters being unable to be recovered promptly, thus significantly affecting the final performance. Conversely, although capable of accelerating the recovery process, higher learning rates may introduce instability in the model, ultimately leading to a decline in performance.

**Optimal Recovery Interval.** As demonstrated in Figure 5, our ablation study investigates the impact of the parameter recovery interval K on model performance, determining that $K = 3$ is the optimal value. The analysis indicates that different parameter settings can significantly affect the model's performance. When the recovery interval is too short, the method fails to obtain sufficient parameter trajectory samples, leading to the ineffectiveness of pseudo-gradient decomposition. Conversely, when the recovery interval is too long, timely parameter recovery is hindered, resulting in performance degradation. The findings suggest that optimal performance can be achieved by appropriately combining parameter updates and recovery, effectively balancing adaptability and stability during model learning.

## 5. Conclusion

This work proposes the PURA framework for test-time adaption in RGBT tracking. It is worth emphasizing that this study introduces TTA in the field of RGB-T tracking, aiming to address the challenges posed by distribution shifts. By leveraging information from test samples, PURA updates a small number of model parameters, thereby accommodating distribution changes. Additionally, the parameter decomposition recovery based on pseudo-gradient decomposition significantly recovers critical parameters in the principal update direction, thereby maintaining model stability. Our adaptive momentum scaling adjusts the update momentum based on the consistency of principal component directions, preventing unstable updates caused by excessively high or low momentum. The experimental results fully verify the significant improvement effect of PURA in various domain shift scenarios.

## Acknowledgements

This work was supported by the National Natural Science Foundation of China (62222302, U22B2055, 62303046).

## References


[1] Yifan Bai, Zeyang Zhao, Yihong Gong, and Xing Wei. Artrackv2: Prompting autoregressive tracker where to look and how to describe. In *CVPR*, pages 19048–19057, 2024. 2

[2] Malik Boudiaf, Romain Mueller, Ismail Ben Ayed, and Luca Bertinetto. Parameter-free online test-time adaptation. In *CVPR*, pages 8344–8353, 2022. 2, 3

[3] Yujue Cai, Xiubao Sui, Guohua Gu, and Qian Chen. Learning modality feature fusion via transformer for rgbt-tracking. *Infrared Physics & Technology*, 133:104819, 2023. 6

[4] Bing Cao, Junliang Guo, Pengfei Zhu, and Qinghua Hu. Bidirectional adapter for multimodal tracking. In *AAAI*, pages 927–935, 2024. 2

[5] Yutao Cui, Cheng Jiang, Limin Wang, and Gangshan Wu. Mixformer: End-to-end tracking with iterative mixed attention. In *CVPR*, pages 13608–13618, 2022. 2, 5

[6] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. In *ICLR*, 2021. 5

[7] Heng Fan, Liting Lin, Fan Yang, Peng Chu, Ge Deng, Sijia Yu, Hexin Bai, Yong Xu, Chunyuan Liao, and Haibin Ling. Lasot: A high-quality benchmark for large-scale single object tracking. In *CVPR*, pages 5374–5383, 2019. 6

[8] Mingzheng Feng and Jianbo Su. Learning multi-layer attention aggregation siamese network for robust rgbt tracking. *TMM*, 26:3378–3391, 2024. 6

[9] Angus Galloway, Anna Golubeva, Thomas Tanay, Medhat Moussa, and Graham W Taylor. Batch normalization is a cause of adversarial vulnerability. In *ICML Workshop*. 1

[10] Junyu Gao, Tianzhu Zhang, and Changsheng Xu. Graph convolutional tracking. In *CVPR*, pages 4649–4659, 2019. 2

[11] Junyu Gao, Mengyuan Chen, and Changsheng Xu. Vectorized evidential learning for weakly-supervised temporal action localization. *TPAMI*, 45(12):15949 – 15963, 2023. 1

[12] Junyu Gao, Xuan Yao, and Changsheng Xu. Fast-slow test-time adaptation for online vision-and-language navigation. In *ICML*, 2024. 2, 3

[13] Junyu Gao, Mengyuan Chen, and Changsheng Xu. Learning probabilistic presence-absence evidence for weakly-supervised audio-visual event perception. *TPAMI*, 2025. 1

[14] Xiaojun Hou, Jiazheng Xing, Yijie Qian, Yaowei Guo, Shuo Xin, Junhao Chen, Kai Tang, Mengmeng Wang, Zhengkai Jiang, Liang Liu, et al. Sdstrack: Self-distillation symmetric adapter learning for multi-modal visual object tracking. In *CVPR*, pages 26551–26561, 2024. 2

[15] Lianghua Huang, Xin Zhao, and Kaiqi Huang. Got-10k: A large high-diversity benchmark for generic object tracking in the wild. *TPAMI*, 43(5):1562–1577, 2019. 6

[16] Tianrui Hui, Zizheng Xun, Fengguang Peng, Junshi Huang, Xiaoming Wei, Xiaolin Wei, Jiao Dai, Jizhong Han, and Si Liu. Bridging search region interaction with template for rgb-t tracking. In *CVPR*, pages 13630–13639, 2023. 2, 5, 6

[17] Sergey Ioffe. Batch normalization: Accelerating deep network training by reducing internal covariate shift. *arXiv preprint arXiv:1502.03167*, 2015. 2, 3

[18] Simiao Lai, Chang Liu, Jiawen Zhu, Ben Kang, Yang Liu, Dong Wang, and Huchuan Lu. Mambavt: Spatio-temporal contextual modeling for robust rgb-t tracking. *CoRR*, 2024. 5, 6

[19] Chenglong Li, Hui Cheng, Shiyi Hu, Xiaobai Liu, Jin Tang, and Liang Lin. Learning collaborative sparse representation for grayscale-thermal tracking. *TIP*, 25(12):5743–5756, 2016. 5, 6

[20] Chenglong Li, Nan Zhao, Yijuan Lu, Chengli Zhu, and Jin Tang. Weighted sparse representation regularized graph learning for rgb-t object tracking. In *ACM MM*, pages 1856–1864, 2017. 5, 6

[21] Chenglong Li, Xinyan Liang, Yijuan Lu, Nan Zhao, and Jin Tang. Rgb-t object tracking: Benchmark and baseline. *PR*, 96:106977, 2019. 5, 6

[22] Chenglong Li, Lei Liu, Andong Lu, Qing Ji, and Jin Tang. Challenge-aware rgbt tracking. In *ECCV*, pages 222–237. Springer, 2020. 2

[23] Chenglong Li, Wanlin Xue, Yaqing Jia, Zhichen Qu, Bin Luo, Jin Tang, and Dengdi Sun. Lasher: A large-scale high-diversity benchmark for rgbt tracking. *TIP*, 31:392–404, 2021. 7

[24] Yanghao Li, Naiyan Wang, Jianping Shi, Jiaying Liu, and Xiaodi Hou. Revisiting batch normalization for practical domain adaptation. In *ICLR*, 2017. 1

[25] Yanghao Li, Naiyan Wang, Jianping Shi, Xiaodi Hou, and Jiaying Liu. Adaptive batch normalization for practical domain adaptation. *PR*, 80:109–117, 2018. 6, 7

[26] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C Lawrence Zitnick. Microsoft coco: Common objects in context. In *ECCV*, pages 740–755. Springer, 2014. 6

[27] Lei Liu, Chenglong Li, Yun Xiao, Rui Ruan, and Minghao Fan. Rgbt tracking via challenge-based appearance disentanglement and interaction. *TIP*, 33:1753–1767, 2024. 6

[28] Yang Luo, Xiqing Guo, Hui Feng, and Lei Ao. Rgb-t tracking via multi-modal mutual prompt learning. *arXiv preprint arXiv:2308.16386*, 2023. 2, 6

[29] Massimiliano Mancini, Hakan Karaoguz, Elisa Ricci, Patric Jensfelt, and Barbara Caputo. Kitting in the wild through online domain adaptation. In *IROS*, pages 1103–1109, 2018. 2

[30] Jiatian Mei, Dongming Zhou, Jinde Cao, Rencand Nie, and Kangjian He. Differential reinforcement and global collaboration network for rgbt tracking. *Sensors*, 23(7):7301–7311, 2023. 6

[31] Jiatian Mei, Juxiang Zhou, Jun Wang, Jia Hao, Dongming Zhou, and Jinde Cao. Learning multifrequency integration network for rgbt tracking. *Sensors*, 24(9):15517–15530, 2024. 6

[32] Matthias Muller, Adel Bibi, Silvio Giancola, Salman Alsubaihi, and Bernard Ghanem. Trackingnet: A large-scale dataset and benchmark for object tracking in the wild. In *ECCV*, pages 300–317, 2018. 6

[33] Zachary Nado, Shreyas Padhy, D Sculley, Alexander D'Amour, Balaji Lakshminarayanan, and Jasper Snoek.

Evaluating prediction-time batch normalization for robustness under covariate shift. *arXiv preprint arXiv:2006.10963*, 2020. 2

[34] Shuaicheng Niu, Jiaxiang Wu, Yifan Zhang, Yaofo Chen, Shijian Zheng, Peilin Zhao, and Mingkui Tan. Efficient test-time model adaptation without forgetting. In *ICML*, pages 16888–16905. PMLR, 2022. 2, 3, 4, 6, 7

[35] Shuaicheng Niu, Jiaxiang Wu, Yifan Zhang, Zhiquan Wen, Yaofo Chen, Peilin Zhao, and Mingkui Tan. Towards stable test-time adaptation in dynamic wild world. In *ICLR*, 2023. 2, 3

[36] Steffen Schneider, Evgenia Rusak, Luisa Eck, Oliver Bringmann, Wieland Brendel, and Matthias Bethge. Improving robustness against common corruptions by covariate shift adaptation. *NeurIPS*, 33:11539–11551, 2020. 1, 2

[37] Dengdi Sun, Yajie Pan, Andong Lu, Chenglong Li, and Bin Luo. Transformer rgbt tracking with spatio-temporal multimodal tokens. *TCSVT*, 34(11_Part_2):12059–12072, 2024. 6

[38] Antti Tarvainen and Harri Valpola. Mean teachers are better role models: Weight-averaged consistency targets improve semi-supervised deep learning results. *NeurIPS*, 30, 2017. 3

[39] Dequan Wang, Evan Shelhamer, Shaoteng Liu, Bruno Olshausen, and Trevor Darrell. Tent: Fully test-time adaptation by entropy minimization. In *ICLR*, 2021. 2, 3, 6, 7

[40] Hongyu Wang, Xiaotao Liu, Yifan Li, Meng Sun, Dian Yuan, and Jing Liu. Temporal adaptive rgbt tracking with modality prompt. In *AAAI*, pages 5436–5444, 2024. 2, 6

[41] Qin Wang, Olga Fink, Luc Van Gool, and Dengxin Dai. Continual test-time domain adaptation. In *CVPR*, pages 7201–7211, 2022. 2, 3, 4, 6, 7

[42] Zhe Wang, Jake Grigsby, and Yanjun Qi. Pgrad: Learning principal gradients for domain generalization. In *ICLR*, 2023. 1

[43] Svante Wold, Kim Esbensen, and Paul Geladi. Principal component analysis. *Chemometrics and intelligent laboratory systems*, 2(1-3):37–52, 1987. 4

[44] Yun Xiao, Mengmeng Yang, Chenglong Li, Lei Liu, and Jin Tang. Attribute-based progressive fusion network for rgbt tracking. In *AAAI*, pages 2831–2838, 2022. 6

[45] Botao Ye, Hong Chang, Bingpeng Ma, Shiguang Shan, and Xilin Chen. Joint feature learning and relation modeling for tracking: A one-stream framework. In *ECCV*, pages 341–357. Springer, 2022. 2, 5

[46] Zhirong Zeng, Xiaotao Liu, Meng Sun, Hongyu Wang, and Jing Liu. Cross fusion rgb-t tracking with bi-directional adapter. *CoRR*, 2024. 6

[47] Lu Zhang, Siqi Zhang, Xu Yang, Hong Qiao, and Zhiyong Liu. Unseen object instance segmentation with fully test-time rgb-d embeddings adaptation. In *ICRA*, pages 4945–4952. IEEE, 2023. 2, 3

[48] Pengyu Zhang, Jie Zhao, Chunjuan Bo, Dong Wang, Huchuan Lu, and Xiaoyun Yang. Jointly modeling motion and appearance cues for robust rgb-t tracking. *TIP*, 30:3335–3347, 2021. 2

[49] Pengyu Zhang, Jie Zhao, Dong Wang, Huchuan Lu, and Xiang Ruan. Visible-thermal uav tracking: A large-scale benchmark and new baseline. In *CVPR*, pages 8886–8895, 2022. 6

[50] Tianlu Zhang, Hongyuan Guo, Qiang Jiao, Qiang Zhang, and Jungong Han. Efficient rgb-t tracking via cross-modality distillation. In *CVPR*, pages 5404–5413, 2023. 6

[51] Xin Zhang and Ying-Cong Chen. Adaptive domain generalization via online disagreement minimization. *TIP*, 32:4247–4258, 2023. 2, 3

[52] Bowen Zhao, Chen Chen, and Shu-Tao Xia. Delta: Degradation-free fully test-time adaptation. In *ICLR*, 2023. 2

[53] Aihua Zheng, Juncong Liu, Zi Wang, Lili Huang, Chenglong Li, and Bing Yin. Visible-infrared person re-identification via specific and shared representations learning. *Visual Intelligence*, 1(1):29, 2023. 1

[54] Jiawen Zhu, Simiao Lai, Xin Chen, Dong Wang, and Huchuan Lu. Visual prompt multi-modal tracking. In *CVPR*, pages 9516–9526, 2023. 2, 6